



# Region-based Resource Management and Lexical Exception Handlers in Continuation-Passing Style

Philipp Schuster (✉), Jonathan Immanuel Brachthäuser, and Klaus Ostermann

University of Tübingen, Germany  
philipp.schuster@uni-tuebingen.de

**Abstract.** Regions are a useful tool for the safe and automatic management of resources. Due to their scarcity, resources are often limited in their lifetime which is associated with a certain scope. When control flow leaves the scope, the resources are released. Exceptions can non-locally exit such scopes and it is important that resources are also released in this case.

Continuation-passing style is a useful compiler intermediate language that makes control flow explicit. All calls are tail calls and the runtime stack is not used. It can also serve as an implementation technique for control effects like exceptions. In this case throwing an exception means jumping to a continuation which is not the current one.

How is it possible to offer region-based resource management and exceptions in the same language and translate both to continuation-passing style? In this paper, we answer this question. We present a typed language with resources and exceptions, and its translation to continuation-passing style. The translation can be defined modularly for resources and exceptions – the correct interaction between the two automatically arises from simple composition. We prove that the translation preserves well-typedness and semantics.

## 1 Introduction

Regions were originally introduced for the safe and automatic management of memory [33]. Since then, much research extended their usefulness for memory management in different scenarios [9, 12–14]. Regions are also a useful tool for controlling the allocation, release, and use of any kind of scarce resource even when considering memory to be plentiful [19]. Resources are organized into a stack of regions which corresponds to nested scopes in the program. Resources in a region are automatically released when control flow leaves the corresponding scope. A type-and-region system guarantees *resource safety*, *i.e.*, that there is no access to a resource outside of its corresponding scope.

Exceptions allow for non-local exits from scopes. It is important that resources are released not only upon normal return, but also when an exception is thrown. A type-and-effect system statically ensures that certain error conditions do not occur when running a program. In the case of exceptions, for example, we want to guarantee *exception safety*, *i.e.*, every exception is eventually caught.

Some work on regions explicitly caters to exceptions [14, 18, 19, 32]. Still, the interaction between regions, exceptions, and first-class functions is non-trivial. To the best of our knowledge region safety for a language with this combination of features has not yet been formally established.

Continuation-passing style (CPS) is an attractive [2, 8, 17] intermediate representation for programs. Control flow is explicit and many program optimizations amount to simple inlining and beta reduction. CPS can also be an implementation technique for control effects like exceptions [16, 17, 26]. Optimization of programs using these features still amounts to inlining and reduction. In CPS all calls are tail calls. Importantly, *there is no runtime stack* that a thrown exception unwinds. Instead, throwing an exception means jumping to a continuation other than the current one.

A CPS translation (from a source to a target language in CPS) must of course be correct, *i.e.* preserve the semantics of the source language. Ideally, the target language is also typed, and the translation takes well-typed terms to well-typed terms. Moreover, when we translate a source program with exceptions to CPS, well-typedness of the target term should also entail exception safety. However, there is not yet a single CPS translation for both exceptions and resource management in the same language. Moreover, since in CPS there is no stack, it is not possible to run cleanup actions during unwinding. Therefore it is not clear how such a combination in CPS could guarantee proper release of resources when an exception is thrown.

We present an intermediate language  $\Lambda_p$  with resources and exceptions. It has a type-and-effect system keeping track of regions to model both: the lifetime of resources as well as the scope of exception handlers. We define its operational semantics as an instrumented [23] abstract machine, which manipulates a runtime stack. We prove progress (Theorem 1) and preservation (Theorem 2) for this semantics in the proof assistant Coq. Resource safety (Corollary 1) and exception safety (Corollary 2) follow as corollaries. To our knowledge, this is the first proof of safety for a language with region-based resource management, exceptions, and first-class functions.

We define a CPS translation from  $\Lambda_p$  to **System F** with base types and primitive operations. The translation takes well-typed terms to well-typed terms (Theorem 3). We implemented the translation as a shallow embedding into the dependently typed language Idris 2. It does not use any special runtime constructs, neither for regions nor for exceptions. The translation is correct: translated terms simulate the abstract machine semantics step-wise (Theorem 4). This entails resource safety and exception safety for CPS translated terms.

Our key technical idea is to understand regions as describing the runtime stack. In the operational semantics, language constructs for resources and exceptions push freshly generated markers onto the runtime stack. At runtime, a region stands for the concrete list of markers on the stack and subregioning evidence stands for the concrete difference between two such lists. In CPS there is no stack. Under our CPS translation, regions are answer types [30], and subregioning evidence terms are answer-type coercing functions. They move from one

region to another one. This allows us to define the CPS translations of resource management and exceptions separately while having them interact correctly.

The rest of the paper is organized as follows. In Section 2, we introduce the main ideas behind our language  $\Lambda_\rho$ . In Section 3, we formally present  $\Lambda_\rho$ . We start with a base language with type-level region tracking and term-level subregioning evidence. We gradually extend this base language with region-based resource management and exceptions. In Section 4, we define the CPS translation for  $\Lambda_\rho$  to System F. We do so gradually, first for the base language, then for resources, then for exceptions. In Section 5 we compare to related work and in Section 6 we summarize the key ideas and outline future work.

## 2 Overview

Here, we provide an informal overview of our main ideas and the language  $\Lambda_\rho$ . We start by re-iterating how regions are used for resource management. We then introduce exceptions and show how we translate them to CPS. Finally, we combine resources and exceptions and demonstrate how our translation reveals information about the use of resources in the presence of non-local exits.

### 2.1 Regions for Resources

As a first example, let us see how regions can be used to manage file handles in  $\Lambda_\rho$ . Our type system follows Fluet and Morrisett [12] and Kiselyov and Shan [19] with some minor differences.

*Example 1.* Consider the following simple example, which copies the first line of a file "input" into a file "output" and additionally inserts a line at the beginning and a line at the end of the output file. Both files are automatically closed and any attempt, accidental or not, to use them after they are closed will fail.

```
pool { [r1](p1 : Pool r1, l1 : r1  $\sqsubseteq$  Top)  $\Rightarrow$ 
  val out: File r1 = open(p1, "output", 0);
  writeln(out, "start", 0);
  pool { [r2](p2 : Pool r2, l2 : r2  $\sqsubseteq$  r1)  $\Rightarrow$ 
    val in: File r2 = open(p2, "input", 0);
    val firstLine = readln(in, 0);
    writeln(out, firstLine, l2)
  };
  writeln(out, "end", 0);
  return ()
}
```

We use a `pool { ... }` statement to create a fresh resource pool. A pool is a reference to a list of open files. All files in this list are automatically closed when control flow leaves the enclosed block. The `pool` statement introduces a *region variable*  $r_1$ , a *pool variable*  $p_1$  and *subregioning evidence*  $l_1$ . We then open the file "output" in pool  $p_1$ . In our type system, every statement is checked

in a region. The overall statement is checked in the top-level region `Top`. The enclosed block is checked in region `r1`. When we open a file, we have to explicitly pass evidence that the current region is a subregion of the pool's region. In this example, we pass the reflexivity evidence  $\emptyset : r1 \sqsubseteq r1$ . We create a second pool `p2` in a second region `r2`, which is clearly inside of `r1`. This fact is witnessed by the evidence variable `l2`. When we write to the output file, we have to provide evidence that the current region `r2` is inside of the file's region `r1`. We provide  $l2 : r2 \sqsubseteq r1$ .

For this simple example, after applying our CPS translation and some beta reduction we get the following straight-line code.

```

λk.
  let p1 = createPool ();
  let out = openFile p1 "output";
  writeLine out "start";
  let p2 = createPool ();
  let in = openFile p2 "input";
  let firstLine = readLine in;
  writeLine out firstLine;
  releasePool p2;
  writeLine out "end";
  releasePool p1;
  k ()

```

The original program did not contain any interesting control flow and our CPS translation results in a sequence of primitive operations. There is no overhead for protecting resources when no exception is thrown. Later we will see how we clean up resources when there are exceptions. But first, let us look at our CPS translation of exceptions.

## 2.2 Regions for Exception Handlers

Exceptions abort the current computation to an exception handler. An exception that is thrown while the corresponding handler is not on the stack results in an error condition that we statically prevent from happening. In  $A_\rho$ , we use the same mechanism for resources and exceptions and enforce *exception safety* in terms of regions: in order to throw to an exception handler, we require evidence that the corresponding handler is still on the stack.

Exceptions in  $A_\rho$  are *lexically scoped*: the connection between a thrown exception and its handler is established by a variable that stands for this very handler [5, 6, 35, 36]. This style of exceptions is in contrast to traditional exceptions, which are caught by the dynamically closest handler. Lexical exception handlers have advantages when reasoning about higher-order functions. Operationally, each `try` statement generates a fresh marker at runtime and pushes a catch frame with this marker onto the stack. We explicitly pass these markers as values of type `Catch r`. For example, consider the following program.

*Example 2.* The function `safeDiv` divides two numbers, but throws an exception when the second number is zero.

```
def safeDiv[r](x : Int, y : Int, e : Catch r) at r {
  if (y == 0) { throw(e, 0) }
  else { return (x / y) }
}
```

In addition to the two parameters `x` and `y`, the function `safeDiv` receives a catch marker `e`. When `y` is zero we throw to `e`. For this to be safe we need to guarantee that we only throw to `e` in the dynamic extent of the corresponding exception handler. But this is the very same problem we had with pools. So we use the very same solution: When we throw to a catch frame of type `Catch r` we have to provide evidence that the current region is a subregion of the catch's region, in this example  $0 : r \sqsubseteq r$ .

The function `safeDiv` is *region polymorphic*. It abstracts over a region variable `r`. It is also annotated to run in the region `r`. To handle the exception we use our `safeDiv` function as follows.

```
try { [r1](e1 : Catch r1, l1 : r1 ⊆ Top) ⇒
  safeDiv[r1](5, 0, e1)
} catch { return 0 }
```

Very much like the `pool` statement, the exception handler introduces a region variable `r1`, a handler `e1`, and subregioning evidence `l1`. In the call to `safeDiv`, we instantiate the region variable `r` to `r1` and pass the exception handler `e1`. The example illustrates that we can guarantee exception safety by the very same mechanism we use for region safety.

When we translate this program to CPS, inline the function `safeDiv`, and after applying beta reduction and commuting conversions we get the following:

$$\lambda k_2. \text{if } (0 \equiv 0) \text{ then } k_2 0 \text{ else } k_2 (5 / 0)$$

When we translate programs to CPS, control flow becomes explicit. This is also true in the presence of control effects like exceptions. Because of this, optimizing programs in CPS amounts to beta reduction. How then can we achieve the same in the presence of resources *and* exceptions?

## 2.3 Combining Resources and Exceptions

Consider the following simple program that mixes pools and exceptions.

*Example 3.* We install an exception handler and create two resource pools. We open a file in the inner pool, open a file in the outer pool, and then throw an exception.

```

try { [r1](e1 : Catch r1, l1 : r1  $\sqsubseteq$  Top)  $\Rightarrow$ 
  pool { [r2](p2 : Pool r2, l2 : r2  $\sqsubseteq$  r1)  $\Rightarrow$ 
    pool { [r3](p3 : Pool r3, l3 : r3  $\sqsubseteq$  r2)  $\Rightarrow$ 
      open(p3, "input", 0);
      open(p2, "output", l3);
      throw(e1, l3  $\oplus$  l2)
    }
  }
} catch { return 1 }

```

To open files into pools, we have to provide evidence, as before. To throw an exception to the outer handler  $e1$ , we provide evidence that region  $r3$  is inside of  $r1$ . We *compose* evidence variables  $l3 \oplus l2$ , to get evidence of type  $r3 \sqsubseteq r1$ .

This program, after CPS translation, reduces to the following program. The exception handler is known and will be eliminated. Again, simplifying control flow amounts to beta reduction as usual in CPS.

```

λk.
  let p2 = createPool ();
  let p3 = createPool ();
  openFile p3 "input";
  openFile p2 "output";
  releasePool p3;
  releasePool p2;
  k 1

```

In our framework, these simplifications of control flow also correctly account for proper creation and release of resources. We can blindly reduce the translated program without any extra considerations.

## 2.4 First-Class Functions

The language  $\Lambda_\rho$  supports first-class functions. For example, consider the following program which factors out a common pattern as a higher-order function.

```

def withFile[r0](path: String, f: [r](File r, r  $\sqsubseteq$  r0)  $\rightarrow$  r Unit) at r0 {
  pool { [r1](p1: Pool r1, l1: r1  $\sqsubseteq$  r0)  $\Rightarrow$ 
    val file = open(p1, path, 0);
    f[r1](file, l1)
  }
}

```

The function `withFile` is region polymorphic. It abstracts over the region  $r0$  it can be used in. The function  $f$  must be region polymorphic too, because we use it under a new region  $r1$ . We instantiate its region parameter with  $r1$  and pass evidence  $l1$ . It would be possible to write `withFile` with the following signature:

```
withFile : [r0](path: String, f: [r](File r)  $\rightarrow$  r Unit)  $\rightarrow$  r0 Unit
```

Here, the function parameter  $f$  would not receive any evidence. This variant of `withFile` would be less useful, as  $f$  could not access any resources from outside of the call-site of `withFile`.

### 3 A Language with Regions, Resources, and Exceptions

In this section, we formally present  $\Lambda_\rho$  and its operational semantics. We will introduce  $\Lambda_\rho$  step-by-step starting with a base language with support for type-level region tracking but no interesting term-level features that make use of them. We then add resource pools, exceptions, and finally consider the combination of the two. The operational semantics is given in terms of an abstract machine that manipulates a runtime stack. In Section 4, we present a CPS translation of  $\Lambda_\rho$ , following the same incremental development.

The paper is accompanied by a mechanized formalization of  $\Lambda_\rho$  and its operational semantics in the Coq theorem prover [3], including the usual theorems of Progress (Theorem 1) and Preservation (Theorem 2). Resource- and exception safety follow as corollaries: whenever we use a resource (like a file) it is live (Corollary 1), and whenever we throw an exception the corresponding handler is on the stack (Corollary 2).

Our operational semantics will push freshly generated markers onto the runtime stack. A region is the list of concrete *markers on the stack* and evidence is the list of markers that is the *difference* between two such lists. Although they do not play any role computationally, for our proofs we will substitute these lists for region variables and evidence variables at runtime. Our typing rule for runtime evidence then makes proving region safety and exception safety possible.

#### 3.1 Syntax

Figure 1 defines the syntax of the core of  $\Lambda_\rho$ . We use fine-grain call-by-value [22] and syntactically distinguish between statements, which can have effects, and pure expressions.

Function values (*i.e.*,  $\{ [\bar{r}](\overline{x : \tau}) \text{ at } \rho \Rightarrow s \}$ ) abstract over a list of type-level region parameters (*i.e.*,  $\bar{r}$ ), and a list of term-level parameters (*i.e.*,  $\overline{x : \tau}$ ). Each function is defined to run exactly in a region  $\rho$ , but otherwise functions are unsurprising. Since our focus is on the interaction between regions and control effects, we omit type abstraction from this presentation. Our mechanization includes type polymorphism, which is orthogonal to the rest of the calculus. We define the following short-hand notation for named function definitions:

$$\text{def } f[\bar{r}](\overline{x : \tau}) \text{ at } \rho \{ s_0 \}; s \quad \doteq \quad \text{val } f = \text{return } \{ [\bar{r}](\overline{x : \tau}) \text{ at } \rho \Rightarrow s_0 \}; s$$

The list of region parameters scopes over the parameter types, the return type, the annotated region  $\rho$ , and the body  $s$  of the function. We apply functions to a list of regions  $\bar{\rho}$  and a list of arguments  $\bar{e}$ .

We introduce two additional concepts: type-level regions and term-level evidence. Type-level regions  $\rho$  are region variables  $r$  or the top-level region  $\top$ .

**Terms:**

## Statements

$s ::= \text{val } x = s; s$     sequencing  
 $\quad \mid \text{return } e$     returning  
 $\quad \mid e[\bar{\rho}](\bar{e})$     application

## Expressions

$e, i ::= x, f, l, \dots$     variables  
 $\quad \mid v$     values  
 $\quad \mid \mathbb{0}$     reflexivity ev.  
 $\quad \mid e \oplus e$     transitivity ev.

## Values

$v ::= () \mid 0 \mid 1 \mid \dots \mid \text{true} \mid \dots$     primitives  
 $\quad \mid \{ [\bar{r}](\bar{x} : \bar{\tau}) \text{ at } \rho \Rightarrow s \}$     closures

**Types:**

## Types

$\tau ::= \text{Int} \mid \text{Bool} \mid \dots$     primitives  
 $\quad \mid \forall[\bar{r}] (\bar{\tau}) \rightarrow^\rho \tau$     functions  
 $\quad \mid \rho \sqsubseteq \rho$     evidence

## Regions

$\rho ::= r$     region variable  
 $\quad \mid \top$     toplevel region

**Environments:**

$\Gamma ::= \emptyset$     empty env.  
 $\quad \mid \Gamma, r$     region binding  
 $\quad \mid \Gamma, x : \tau$     value binding

**Names:**

$x, y, l ::= x \mid y \mid l \dots$     value variables     $r ::= r \mid s \mid \dots$     region variables

**Fig. 1.** Syntax of the core of  $\Lambda_\rho$ .

Intuitively, the top-level region denotes the bottom part of the runtime stack. Term-level evidence expressions are either the empty evidence  $\mathbb{0}$  witnessing reflexivity of subregioning, or the composition of evidence  $e \oplus e$  witnessing transitivity of subregioning. By convention, we use the meta-variables  $f$  and  $l$  to stand for variables of function type and evidence type respectively, and we use the meta-variable  $i$  to stand for expressions of evidence type.

**3.2 Typing**

Figure 2 defines the typing of core  $\Lambda_\rho$ . We type statements and expressions with different judgement forms. While both are typed in an environment  $\Gamma$  containing value and region bindings, only statements are typed in a given region  $\rho$ . Statements may perform effectful (that is, *serious* in the terminology of Reynolds [24]) computation, which is only safe in specific regions. In contrast, expressions are pure (that is, *trivial*) and can be evaluated independent of any region.

*Typing of Statements* Rule VAL types sequencing of statements. We type the two statements  $s_0$  and  $s$  in the same region  $\rho$  of the compound statement. Returning a result of a computation (rule RET) can be typed in any region. In rule APP, we apply a function  $e_0$  to a list of regions  $\bar{\rho}$  and to a list of arguments  $\bar{e}$ . The type of  $e_0$  is a function type in a region  $\rho_0$ . The overall statement is typed in a region  $\rho$ . The premise  $\rho = \rho_0[\bar{r} \mapsto \bar{\rho}]$  requires that, after substituting regions  $\bar{\rho}$  for the region variables  $\bar{r}$  both have to syntactically be the same. Note that we do not have any implicit or explicit subtyping of function types here or elsewhere. All region subtyping exclusively occurs through the passing of subregioning evidence.



**Statement Typing:**

$$\boxed{\begin{array}{c} \Gamma \vdash \rho \vdash s : \tau \\ \uparrow \quad \uparrow \quad \uparrow \quad \downarrow \end{array}}$$

$$\frac{\Gamma \vdash e_0 : \forall[\bar{\tau}](\bar{\tau}) \rightarrow^{\rho_0} \tau_0 \quad \overline{\Gamma \vdash e : \tau[\bar{\tau} \mapsto \bar{\rho}]}}{\Gamma \vdash \rho \vdash e_0[\bar{\rho}](\bar{e}) : \tau_0[\bar{\tau} \mapsto \bar{\rho}]} \text{ [APP]}$$

$$\frac{\Gamma \vdash \rho \vdash s_0 : \tau_0 \quad \Gamma, x_0 : \tau_0 \vdash \rho \vdash s : \tau}{\Gamma \vdash \rho \vdash \mathbf{val} x_0 = s_0; s : \tau} \text{ [VAL]} \quad \frac{\Gamma \vdash e : \tau}{\Gamma \vdash \rho \vdash \mathbf{return} e : \tau} \text{ [RET]}$$

**Expression Typing:**

$$\boxed{\begin{array}{c} \Gamma \vdash e : \tau \\ \uparrow \quad \uparrow \quad \downarrow \end{array}}$$

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \text{ [VAR]} \quad \frac{}{\Gamma \vdash n : \text{Int}} \text{ [LIT]} \quad \frac{\Gamma, \bar{\tau}, \bar{x} : \bar{\tau} \vdash \rho \vdash s_0 : \tau_0}{\Gamma \vdash \{ [\bar{\tau}](\bar{x} : \bar{\tau}) \mathbf{at} \rho \Rightarrow s_0 \} : \forall[\bar{\tau}](\bar{\tau}) \rightarrow^{\rho} \tau_0} \text{ [FUN]}$$

$$\frac{}{\Gamma \vdash \mathbf{0} : \rho \sqsubseteq \rho} \text{ [REFLEXIVE]} \quad \frac{\Gamma \vdash e : \rho \sqsubseteq \rho' \quad \Gamma \vdash e' : \rho' \sqsubseteq \rho''}{\Gamma \vdash e \oplus e' : \rho \sqsubseteq \rho''} \text{ [TRANSITIVE]}$$

**Fig. 2.** Type system of the core of  $\Lambda_\rho$ .

*Typing of Expressions* The typing rules for variables VAR and primitives LIT are standard. Rule FUN types functions. We type the body of the function  $s_0$  in an environment extended with the region parameters  $\bar{\tau}$  and value parameter types  $\bar{x} : \bar{\tau}$ . Every function is annotated with a region  $\rho$  that specifies *exactly* the region it will have to be called in. This region  $\rho$  is also the region we type the body  $s_0$  in. The region parameters  $\bar{\tau}$  may appear in the parameter types, the return type, the function's region  $\rho$ , and body  $s_0$ . This allows us to write *region-polymorphic functions* that can run in any region. Value parameters of evidence type allow us to write region-polymorphic functions that are *constrained* to run in a subregion that meets these constraints.

Reflexivity evidence  $\mathbf{0}$  witnesses that every region is nested within itself, and evidence  $e \oplus e'$  witnesses the transitivity of nesting, which is reflected in their typing rules. We require the composition of evidence to be associative.

**3.3 Operational Semantics**

Figure 3 presents the operational semantics of core  $\Lambda_\rho$ . A machine state  $\langle s \parallel K \rangle$  consists of the statement  $s$  under evaluation and the runtime stack  $K$ . For the core of  $\Lambda_\rho$ , the stack  $K$  is a list of frames of the form  $\mathbf{val} x = \square; s$ . The reduction rules are mostly standard. The first rule (*return*) returns to the next frame on the stack. The second rule (*push*) focuses on  $s_0$  and pushes a frame on the stack. Finally, rule (*call*) performs reduction by simultaneously substituting region arguments  $\bar{\rho}$  for region variables  $\bar{\tau}$  and trivial expressions  $\bar{e}$  for term parameters  $\bar{x}$ . Region parameters, the annotated region  $\rho$ , and evidence terms

**Syntax of the Abstract Machine:**

| Machine States                        | Stacks                      | Frames                              |
|---------------------------------------|-----------------------------|-------------------------------------|
| $M ::= \langle s \parallel K \rangle$ | $K ::= \bullet \mid F :: K$ | $F ::= \mathbf{val} x = \square; s$ |

**Machine Steps:**

|                                                                                                                                     |                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| $(return) \langle \mathbf{return} e \parallel \mathbf{val} x = \square; s :: K \rangle$                                             | $\rightarrow \langle s[x \mapsto e] \parallel K \rangle$                                             |
| $(push) \langle \mathbf{val} x = s_0; s \parallel K \rangle$                                                                        | $\rightarrow \langle s_0 \parallel \mathbf{val} x = \square; s :: K \rangle$                         |
| $(call) \langle \{ [\bar{r}] (\bar{x} : \bar{\tau}) \mathbf{at} \rho \Rightarrow s_0 \} [\bar{\rho}] (\bar{e}) \parallel K \rangle$ | $\rightarrow \langle s_0 [\bar{r} \mapsto \bar{\rho}] [\bar{x} \mapsto \bar{e}] \parallel K \rangle$ |

**Extended Syntax:**

|                         |                |
|-------------------------|----------------|
| $v ::= \dots \mid w$    | evidence value |
| $\rho ::= \dots \mid u$ | runtime region |

**Runtime Regions and Evidence:**

|                 |                 |
|-----------------|-----------------|
| $w ::= \bullet$ | evidence values |
| $u ::= \bullet$ | runtime regions |

**Runtime Region of Stack:**

|                                                         |                            |
|---------------------------------------------------------|----------------------------|
| $\mathcal{R}[\![ \cdot ]\!]$                            | $: K \rightarrow u$        |
| $\mathcal{R}[\![ \bullet ]\!]$                          | $= \bullet$                |
| $\mathcal{R}[\![ \mathbf{val} x = \square; s :: K ]\!]$ | $= \mathcal{R}[\![ K ]\!]$ |

**Evaluation of Evidence:**

|                                       |                                                          |
|---------------------------------------|----------------------------------------------------------|
| $\mathcal{V}[\![ \cdot ]\!]$          | $: e \rightarrow w$                                      |
| $\mathcal{V}[\![ \mathbf{0} ]\!]$     | $= \bullet$                                              |
| $\mathcal{V}[\![ e_1 \oplus e_2 ]\!]$ | $= \mathcal{V}[\![ e_1 ]\!] ++ \mathcal{V}[\![ e_2 ]\!]$ |
| $\mathcal{V}[\![ w ]\!]$              | $= w$                                                    |

**Fig. 3.** Abstract machine semantics of core  $\Lambda_\rho$ .

are operationally irrelevant. As already mentioned, we need them to maintain invariants in our proofs.

The core of  $\Lambda_\rho$ , as presented, does not yet contain features with interesting operational behavior. While we can abstract over regions, eventually all region variables will be instantiated with the top-level region and evidence will always be the trivial evidence.

Figure 3 also defines runtime regions and evidence values in core  $\Lambda_\rho$ . We extend the syntax of values with evidence values  $w$ , and the syntax of regions with runtime regions  $u$ . Both are empty lists  $\bullet$  for now. In the next two sections, we will extend their syntax to be lists for markers  $h$ . The toplevel region  $\top$  is the empty list runtime region  $\bullet$ .

To connect type-level regions  $\rho$  with the concrete runtime stack  $K$ , we define a semantic function  $\mathcal{R}[\![ \cdot ]\!]$ , which computes the runtime region of the current stack. In core  $\Lambda_\rho$ , the only possible runtime region is the empty list. To give meaning to evidence expressions, we define a semantic function  $\mathcal{V}[\![ \cdot ]\!]$ . Currently the only possible evidence value is the empty list.

**3.4 Resource Pools**

In this subsection, we add statements for region-based resource management to  $\Lambda_\rho$ . As in the introduction, we use files as an example for resources. Figure 4

**Syntax:**

| Statements                                  |                   | Types              |
|---------------------------------------------|-------------------|--------------------|
| $s ::= \dots$                               |                   | $\tau ::= \dots$   |
| <b>pool</b> $\{ [r](x, l) \Rightarrow s \}$ | new resource pool | <b>Pool</b> $\rho$ |
| <b>open</b> $(e, e_0, i)$                   | open file         | <b>File</b> $\rho$ |
| <b>readln</b> $(e, i)$                      | read contents     |                    |

**Typing Rules:**

$$\begin{array}{c}
\frac{\Gamma, r, x : \text{Pool } r, l : r \sqsubseteq \rho \vdash r \vdash s : \tau}{\Gamma \vdash \rho \vdash \text{pool} \{ [r](x, l) \Rightarrow s \} : \tau} [\text{POOL}] \quad \frac{\Gamma \vdash e : \text{File } \rho' \quad \Gamma \vdash i : \rho \sqsubseteq \rho'}{\Gamma \vdash \rho \vdash \text{readln}(e, i) : \text{String}} [\text{READ}] \\
\\
\frac{\Gamma \vdash e : \text{Pool } \rho' \quad \Gamma \vdash e_0 : \text{String} \quad \Gamma \vdash i : \rho \sqsubseteq \rho'}{\Gamma \vdash \rho \vdash \text{open}(e, e_0, i) : \text{File } \rho'} [\text{OPEN}]
\end{array}$$

**Fig. 4.** Syntax and typing rules of resource pools.**Syntax of Frames:**

$F ::= \dots \mid \# \text{pool}_h \{ \square \}$  resource pool frame

**Machine Steps:**

$$\begin{array}{l}
(\text{release}) \\
\langle \text{return } e \parallel \# \text{pool}_h \{ \square \} :: K \rangle \rightarrow \langle \text{return } e \parallel K \rangle \quad \text{do } \text{releasePool}(h) \\
\\
(\text{pool}) \\
\langle \text{pool} \{ [r](x, l) \Rightarrow s_0 \} \parallel K \rangle \rightarrow \langle s_0[r \mapsto u][x \mapsto h][l \mapsto w] \parallel \# \text{pool}_h \{ \square \} :: K \rangle \\
\text{do } h = \text{createPool}() \quad \text{where } u = \text{po } h :: \mathcal{R}[\llbracket K \rrbracket], \text{ and } w = \text{po } h :: \bullet \\
\\
(\text{open}) \\
\langle \text{open}(h, e, i) \parallel K \rangle \rightarrow \langle \text{return } x \parallel K \rangle \quad \text{when } \text{po } h \text{ in } \mathcal{R}[\llbracket K \rrbracket] \\
\text{do } x = \text{openFile}(h, e) \\
\\
(\text{read}) \\
\langle \text{readln}(p, i) \parallel K \rangle \rightarrow \langle \text{return } x \parallel K \rangle \quad \text{when } \text{po } h \text{ in } \mathcal{R}[\llbracket K \rrbracket] \\
\text{do } x = \text{readLine}(p) \quad \text{where } h = p.\text{getPool}
\end{array}$$

**Runtime Regions and Evidence:**

$h ::= @a5f \mid @4b2 \mid \dots$  markers

$w ::= \dots \mid \text{po } h :: w$  evidence value

$u ::= \dots \mid \text{po } h :: u$  runtime region

**Runtime Region of Stack:**

$\mathcal{R}[\llbracket \# \text{pool}_h \{ \square \} :: K \rrbracket] = \text{po } h :: \mathcal{R}[\llbracket K \rrbracket]$

**Fig. 5.** Abstract machine semantics of pool-based resource management.

**Syntax:**

| Statements |                                                                     | Types  |                          |
|------------|---------------------------------------------------------------------|--------|--------------------------|
| $s$        | $::= \dots$                                                         | $\tau$ | $::= \dots$              |
|            | <b>try</b> $\{ [r](x, l) \Rightarrow s_0 \}$ <b>catch</b> $\{ s \}$ |        | <b>Catch</b> $\rho \tau$ |
|            | <b>throw</b> ( $e, i$ )                                             |        |                          |
|            | handling                                                            |        | throwing                 |

**Typing Rules:**

|                                                                                                    |                                                       |
|----------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| $\Gamma, r, x : \text{Catch } r, l : r \sqsubseteq \rho \vdash s_0 : \tau$                         | $\Gamma \vdash e : \text{Catch } \rho'$               |
| $\Gamma \vdash \rho \vdash s : \tau$                                                               | $\Gamma \vdash i : \rho \sqsubseteq \rho'$            |
| $\Gamma \vdash \rho \vdash \text{try} \{ [r](x, l) \Rightarrow s_0 \} \text{catch} \{ s \} : \tau$ | $\Gamma \vdash \rho \vdash \text{throw}(e, i) : \tau$ |
| [TRY]                                                                                              | [THROW]                                               |

**Fig. 6.** Syntax and typing rules of exceptions.

introduces three additional statement forms, which introduce and eliminate non-trivial evidence to assert that all files are correctly closed. The **pool** statement delimits a new region in which we run the enclosed statement  $s$ . It introduces three variables, a fresh region variable  $r$ , a variable  $x : \text{Pool } r$ , and evidence  $l : r \sqsubseteq \rho$ , witnessing that the fresh region  $r$  is a subregion of the outer region  $\rho$ . The **open** statement receives a pool argument  $e$ , a filename  $e_0$ , and an evidence argument  $i : \rho \sqsubseteq \rho'$  that witnesses that the current region  $\rho$  is nested within the pool's region  $\rho'$ . Rule READ for **readln** statements is similar.

Figure 5 extends the operational semantics. Frames can now be pool frames which contain a marker  $h$ . In rule (*pool*), we allocate a fresh marker  $h$  and push a pool frame onto the stack. In rule (*release*), we pop the pool frame and release the pool  $h$ , closing all associated resources. Our goal is to ensure that all access to marker  $h$  happens between these two steps.

To this end, rules (*open*) and (*read*) dynamically assert that the marker  $h$  is on the current stack  $K$ . Accessing a pool that fails this test would result in a stuck term. As it turns out, the mere existence of evidence  $i$  suffices to show that the assertion always succeeds (Corollary 1).

For our proof of this fact, Figure 5 extends the syntax of runtime regions and evidence. Runtime regions now include lists of pool markers and so do evidence values. The runtime region of a stack  $K$  is the list of markers that have been pushed onto it. We extend the function  $\mathcal{R}[\cdot]$  to extract this list. During execution, region variables  $r$  stand for runtime regions  $u$ . In rule (*pool*) we substitute the runtime region  $\mathbf{po } h :: \mathcal{R}[K]$  for the region variable  $r$  and the singleton list  $\mathbf{po } h :: \bullet$  for the evidence variable  $l$ . Later we will see how the typing rule for evidence values connects type-level runtime regions with the concrete runtime region of the current stack  $K$ .

**3.5 Exceptions**

Figure 6 extends  $\mathcal{A}_\rho$  with two new statement forms. The **try ... catch ...** statement delimits a new region in which we run the enclosed statement  $s_0$ . It introduces

**Syntax of Frames:**

$$F ::= \dots \mid \#catch_h \{ \square \} \{ s \} \quad \text{catch frame}$$
**Machine Steps:**

(*popcatch*)  
 $\langle \text{return } e \parallel \#catch_h \{ \square \} \{ s \} :: K \rangle \rightarrow \langle \text{return } e \parallel K \rangle$

(*try*)  
 $\langle \text{try} \{ [r](x, l) \Rightarrow s_0 \} \text{catch} \{ s \} \parallel K \rangle \rightarrow$   
 $\langle s_0[r \mapsto u][x \mapsto h][l \mapsto w] \parallel \#catch_h \{ \square \} \{ s \} :: K \rangle$   
 do  $h = \text{generateFresh}()$  where  $u = \mathbf{ca} \ h :: \mathcal{R}[\![K]\!]$  and  $w = \mathbf{ca} \ h :: \bullet$

(*throw*)  
 $\langle \text{throw}(h, i) \parallel K \rangle \rightarrow \langle \text{throw}(h, \mathcal{V}[\![i]\!]) \parallel K \rangle$

(*unwind*)  
 $\langle \text{throw}(h, w) \parallel \text{val } x = \square; s :: K \rangle \rightarrow \langle \text{throw}(h, w) \parallel K \rangle$

(*forward*)  
 $\langle \text{throw}(h, \mathbf{ca} \ h' :: w) \parallel \#catch_{h'} \{ \square \} \{ s \} :: K \rangle \rightarrow \langle \text{throw}(h, w) \parallel K \rangle$   
 where  $h \neq h'$

(*catch*)  
 $\langle \text{throw}(h, \bullet) \parallel \#catch_h \{ \square \} \{ s \} :: K \rangle \rightarrow \langle s \parallel K \rangle$

**Runtime Regions and Evidence:**

$$w ::= \dots \mid \mathbf{ca} \ h :: w \quad \text{evidence value}$$

$$u ::= \dots \mid \mathbf{ca} \ h :: u \quad \text{runtime region}$$
**Runtime Region of Stack:**

$$\mathcal{R}[\![\#catch_h \{ \square \} \{ s \} :: K]\!] =$$

$$\mathbf{ca} \ h :: \mathcal{R}[\![K]\!]$$
**Fig. 7.** Abstract machine semantics of exceptions.

three variables, a fresh region variable  $r$ , a variable  $x : \text{Catch } r$ , and an evidence variable  $l : r \sqsubseteq \rho$ , witnessing that the fresh region  $r$  is a subregion of the outer region  $\rho$ . The **throw** statement receives a handler  $e$  to throw to, and evidence  $i$  that the handler's region  $\rho'$  is nested in the current region  $\rho$ .

Figure 7 extends the operational semantics. Frames can now be catch frames with a marker  $h$  and a catch statement  $s$ . In rule (*try*) we generate a fresh marker  $h$  and push a catch frame with this marker and the catch statement onto the stack. The handler  $x$  is this marker  $h$ . In rule (*popcatch*) we pop this catch frame upon normal return. In rule (*throw*) we transition from normal execution to unwinding.  $h$  is a catch marker, and  $\mathcal{V}[\![i]\!]$  evaluates the evidence expression  $i$  to a list of catch markers. In rules (*unwind*) and (*forward*) we unwind the stack frame-by-frame until we find the matching catch frame (*catch*). Because each **try**

### Extended Machine Steps:

$$(free) \quad \langle \mathbf{throw}(h, \mathbf{po} \ h' :: w) \parallel \# \mathbf{pool}_{h'} \{ \square \} :: K \rangle \rightarrow \langle \mathbf{throw}(h, w) \parallel K \rangle$$

$$\text{do } \mathbf{releasePool}(h')$$

**Fig. 8.** Abstract machine semantics of combining resources and exceptions.

statement generates a fresh marker at runtime, and we search for this marker during unwinding, exceptions have generative semantics [5, 6, 35, 36].

Figure 7 extends the syntax of runtime regions and evidence. They now include lists of catch markers. Again, evidence guarantees that unwinding never fails, *i.e.* the corresponding marker is always somewhere on the stack. Remarkably, we pop elements off the evidence value  $w$  in lock-step with popping catch frames off the stack and never get stuck in doing so. We always find the matching catch frame exactly when the evidence value is the empty list. The evidence value precisely reflects the list of markers between the region of the **throw** statement and the region of the **catch** statement. Importantly, this also holds for the combined language  $\Lambda_\rho$  (Corollary 4).

### 3.6 Combining Resource Pools and Exceptions

When we extend the core language with both pools and exceptions, we notice that the machine gets stuck when we would have to unwind through a pool frame. Figure 8 extends the reduction relation with this missing case. When we unwind through a  $\# \mathbf{pool}_{h'}$  frame, we release the pool  $h'$ . In full  $\Lambda_\rho$  regions are lists where the elements are either a pool marker or an exception marker. Evidence is, again, the same. Having to add the rule in Figure 8 shows that under our operational semantics, the two extensions are not orthogonal. We have to explicitly consider their interaction. In Section 4, we define a CPS translation for  $\Lambda_\rho$ . Remarkably, both extensions can be defined separately and the correct interaction automatically arises from their composition. Perhaps more importantly, the resulting terms in CPS can be reduced freely without having to consider the interaction between pools and exceptions.

### 3.7 Metatheory of $\Lambda_\rho$

We started out with core  $\Lambda_\rho$  only supporting regions and subregioning evidence. We then added two extensions, pools and exceptions, first individually, then together to arrive at the full language. Although we use resource pools for files as an example, our approach generalizes to region-based management of any resource. Indeed, in our mechanization, we do not model files and the **pool** statement only pushes and pops the fresh marker. Instead of **open** and **readln** we have a statement **check** with the following typing rule:

$$\frac{\Gamma \vdash e : \mathbf{Pool} \ \rho' \quad \Gamma \vdash i : \rho \sqsubseteq \rho' \quad \Gamma \upharpoonright \rho \vdash s : \tau}{\Gamma \upharpoonright \rho \vdash \mathbf{check}(e, i); s : \tau} [\text{CHECK}]$$

**Stack Typing:**

$$\boxed{\begin{array}{c} \vdash K : \tau \\ \uparrow \quad \uparrow \end{array}}$$

$$\frac{}{\vdash \bullet : \tau} [\text{EXIT}] \quad \frac{x : \tau \mid \mathcal{R}[\![K]\!]\vdash s : \tau_1 \quad \vdash K : \tau_1}{\vdash \text{val } x = \square; s :: K : \tau} [\text{FRAME}]$$

$$\frac{\vdash K : \tau}{\vdash \# \text{pool}_h \{ \square \} :: K : \tau} [\# \text{POOL}] \quad \frac{\emptyset \mid \mathcal{R}[\![K]\!]\vdash s : \tau \quad \vdash K : \tau}{\vdash \# \text{catch}_h \{ \square \} \{ s \} :: K : \tau} [\# \text{CATCH}]$$

**Abstract Machine Typing:**

$$\frac{\emptyset \mid \mathcal{R}[\![K]\!]\vdash s : \tau \quad \vdash K : \tau}{\vdash \langle s \parallel K \rangle \text{ok}} [\text{MACHINE}]$$

**Evidence Value Typing:**

$$\frac{u_0 = w \dashv\vdash u_1}{\emptyset \vdash w : u_0 \sqsubseteq u_1} [\text{EVIDENCE}]$$

**Fig. 9.** Abstract machine typing of  $\Lambda_p$ 

It asserts that the given pool is on the current runtime stack, *i.e.* live, and crashes the program if it is not. Otherwise it continues to execute statement  $s$ . We can safely access resources by first performing a runtime check and then using unsafe primitive operations. For example we would define

**open**( $e, e_0, i$ ) := **check**( $e, i$ ); **openFile**( $e, e_0$ )

As we will see shortly, this check never fails.

*Soundness* We mechanized the formalization of  $\Lambda_p$  in the Coq theorem prover and showed the usual theorems of progress and preservation of the stepping relation on machine states  $M$ .

**Theorem 1 (Progress).**

If  $\vdash M \text{ok}$ , then either  $M \rightarrow M'$  or  $M$  is of the form  $\langle \text{return } e \parallel \bullet \rangle$  for some expression  $e$ .

**Theorem 2 (Preservation).**

If  $\vdash M \text{ok}$  and  $M \rightarrow M'$  then  $\vdash M' \text{ok}$ .

Figure 9 presents the typing rules for the abstract machine. An abstract machine state is well-typed when the statement  $s$  is well-typed in the concrete runtime region of the stack  $K$ . The typing judgement  $\vdash K : \tau$  types stacks  $K$  that expect a value of type  $\tau$ . An evidence value is well-typed when it is the difference between the two runtime regions  $u_0$  and  $u_1$ .

*Properties* The following properties follow directly from progress and preservation. Firstly, whenever we use a pool, it is live. The operational semantics inspects the runtime stack. But since the check always succeeds we do not have to actually perform it.

**Corollary 1 (Resource Safety).**

If  $\langle \text{open}(h, e_0, i) \parallel K \rangle \text{ok}$ , then  $\text{po } h$  is in  $\mathcal{R}[[K]]$ .

Secondly, whenever we throw an exception, the corresponding handler is on the stack. Moreover, as we have seen from the operational semantics, during the search for the correct handler, we encounter precisely the markers that are in the evidence value.

**Corollary 2 (Effect Safety).**

If  $\langle \text{throw}(h, i) \parallel K \rangle \text{ok}$ , then  $\text{ca } h$  is in  $\mathcal{R}[[K]]$ .

Thirdly, every function runs in exactly the runtime region its type requires. In other words, the type-level region  $\rho$  will at runtime stand for the concrete runtime region of the stack this function is called in.

**Corollary 3 (Region Correspondence).**

If  $\langle \{ [\bar{r}](\bar{x} : \bar{\tau}) \text{at } \rho \Rightarrow s_0 \} [\bar{u}](\bar{e}) \parallel K \rangle \text{ok}$ , then  $\rho[\bar{r} \mapsto \bar{u}] = \mathcal{R}[[K]]$ .

Finally, evidence values are exactly the difference between the two regions. This corollary is inspired by the similarly named theorem of Xie et al. [34].

**Corollary 4 (Evidence Correspondence).**

If an evidence value  $w$  has type  $\rho_0 \sqsubseteq \rho_1$ , then  $\rho_0$  and  $\rho_1$  are runtime regions  $u_0$  and  $u_1$  and  $u_0 = w \dashv\vdash u_1$ .

Together, these corollaries make runtime evidence on the one hand and marker frames on the stack on the other hand redundant. The unwinding can *either* use evidence terms, *or* markers on the stack, since the two agree. The operational semantics uses both to establish this fact. The liveness check for pools is redundant since it always succeeds. It only exists to establish this fact.

We could erase evidence terms and only rely on marker frames on the stack. In the next section, we are going to CPS where there is no stack. Therefore we will do the opposite: Erase marker frames and purely rely on evidence terms to have the correct content at runtime. This is possible because of the correspondence between evidence and runtime regions. Ultimately, this allows us to prove that CPS translated terms behave exactly as the operational semantics (Theorem 4).

## 4 Translation of Regions, Pools, and Exceptions to CPS

We now present the translation of  $\Lambda_\rho$  into System F (with file primitives) in CPS. As a result of the translation, the stack  $K$  becomes an evaluation context [10], regions become *answer types*, and evidence terms become *answer-type coercions*. As before, we will define the translations of core  $\Lambda_\rho$  and the two extensions with file pools and exceptions step-by-step. Our translation can serve as a compilation technique for languages with control effects and resources into any language



**Translation of Types:**

$$\begin{aligned}
\mathcal{T}[\text{Int}] &= \text{Int} \\
\mathcal{T}[r] &= r \\
\mathcal{T}[\top] &= \text{Void} \\
\mathcal{T}[\forall(\bar{\tau})(\bar{\tau}) \rightarrow^\rho \tau_0] &= \overline{\forall r. \mathcal{T}[\bar{\tau}] \rightarrow \text{CPS } \mathcal{T}[\rho] \mathcal{T}[\tau_0]} \\
\mathcal{T}[\rho \sqsubseteq \rho'] &= \forall a. \text{CPS } \mathcal{T}[\rho'] a \rightarrow \text{CPS } \mathcal{T}[\rho] a
\end{aligned}$$

**Translation of Expressions:**

$$\begin{aligned}
\mathcal{E}[x] &= x \\
\mathcal{E}[\{[\bar{\tau}](x : \bar{\tau}) \text{ at } \rho \Rightarrow s\}] &= \overline{\lambda r. \lambda x. \mathcal{S}[s]_\rho} \\
\mathcal{E}[\mathbf{0}] &= \lambda a. \lambda m. m \\
\mathcal{E}[e_1 \oplus e_2] &= \lambda a. \lambda m. \mathcal{E}[e_1] a (\mathcal{E}[e_2] a m)
\end{aligned}$$

**Translation of Statements:**

$$\begin{aligned}
\mathcal{S}[\text{val } x = s_0; s_1]_\rho &= \lambda k. \mathcal{S}[s_0]_\rho (\lambda x. \mathcal{S}[s_1]_\rho k) \\
\mathcal{S}[\text{return } e]_\rho &= \lambda k. k (\mathcal{E}[e]) \\
\mathcal{S}[e_0[\bar{\rho}](\bar{e})]_\rho &= \mathcal{E}[e_0] \overline{\mathcal{T}[\bar{\rho}]} \overline{\mathcal{E}[\bar{e}]}
\end{aligned}$$

**Auxiliary Definitions:**

$$\text{CPS } R A = (A \rightarrow R) \rightarrow R$$

**Fig. 10.** Translation from core  $\Lambda_\rho$  to System F.

that supports first-class functions, making it widely applicable. Moreover, as demonstrated by Schuster et al. [26], modeling control effects with CPS can enable compile-time optimizations for significant performance improvements. We implemented the CPS translation of  $\Lambda_\rho$  as a shallow embedding in Idris 2 [7].

#### 4.1 Translation of Core $\Lambda_\rho$

Figure 10 defines the translation of core  $\Lambda_\rho$  to System F. Our translation targets one particular variant of CPS, called *iterated CPS* [11, 25]. Every stack segment, delimited by a marker, is represented by its own continuation argument. That is, in iterated CPS, functions do not receive one but potentially multiple continuations. This will only become relevant in the presence of exceptions (Section 4.3).

**Translation of Types** Base types, such as `Int` are left unchanged by the translation. We translate region variables to type variables in **System F** and the toplevel region to the empty type `Void`. The translation on types shows that the iterated CPS translation is (so far) very similar to the traditional CPS translation. In particular, the auxiliary meta-definition  $\text{CPS } R A$  is defined as the familiar type  $(A \rightarrow R) \rightarrow R$  of computations in CPS with *return type*  $A$  and *answer type*  $R$ . Evidence terms are functions between effectful computations, as can be seen from the translation of evidence types.

**Translation of Terms** As usual in CPS, we translate sequencing of statements to push a frame onto the current continuation  $k$ , that is, the continuation first runs  $s_1$  and then continues with  $k$ . Return statements are translated to tail

**Extended Translation Rules:**

$$\begin{aligned}
\mathcal{T}[\text{Pool } \rho] &= \text{PrimPool} \\
\mathcal{T}[\text{File } \rho] &= \text{PrimFile} \\
\mathcal{S}[\text{pool } \{ [r](x, l) \Rightarrow s_0 \}]_{\rho} &= \\
&\quad \text{RUNPOOL } (\lambda h. (\lambda r. \lambda x. \lambda l. \mathcal{S}[s_0]_r) (\mathcal{T}[\rho]) \ h \ (\text{LIFTPOOL } h)) \\
\mathcal{S}[\text{open}(e, e_0, i)]_{\rho} &= \lambda k. k (\text{openFile } \mathcal{E}[e] \ \mathcal{E}[e_0]) \\
\mathcal{S}[\text{readln}(e, i)]_{\rho} &= \lambda k. k (\text{readLine } \mathcal{E}[e])
\end{aligned}$$

**Auxiliary Definitions:**

$$\begin{aligned}
\text{RUNPOOL} &: (\text{PrimPool} \rightarrow \text{CPS } R \ A) \rightarrow \text{CPS } R \ A \\
\text{RUNPOOL} &= \lambda m. \lambda k. \text{let } h = \text{createPool } (); \ m \ h \ (\lambda x. \text{releasePool } h; \ k \ x) \\
\text{LIFTPOOL } h &: \forall a. \text{CPS } R \ a \rightarrow \text{CPS } R \ a \\
\text{LIFTPOOL } h &= \Lambda a. \lambda m. \lambda k. \text{releasePool } h; \ m \ k
\end{aligned}$$

**Fig. 11.** Translation of  $\Lambda_{\rho}$  with resource pools.

calls of the current continuation. Again, viewing continuations as stacks, this is in accordance with the operational semantics given in Section 3.3. In general, statements with return type  $\tau$  that have to be run in a region  $\rho$  are translated to terms of type  $\text{CPS } \mathcal{T}[\rho] \ \mathcal{T}[\tau]$ . This can for instance be seen in the translation of function types. We translate regions to answer types. Region abstractions are translated to type abstractions and region-polymorphic functions have a polymorphic answer type [30]. We translate evidence expressions to functions that lift a computation to run in a different region. The reflexivity evidence is translated to the polymorphic identity function, and transitivity of evidence amounts to function composition.

In the remainder of this section, we present the rest of the translation of our language with pools and exceptions  $\Lambda_{\rho}$ . Later, we show that the translated code in CPS simulates the operational semantics given in Section 3.

**4.2 Resource Pools**

In Figure 4, we have seen the definition of  $\Lambda_{\rho}$  with resource pools. Figure 11 defines the translation to CPS. As we have seen in Section 3.7, we do not need any runtime checks to prevent markers and files from being used outside of their region. Indeed, in CPS there is no stack, which we could check for markers.

The **pool** statement creates a fresh resource pool. The translation instantiates  $r$  with the outer answer type  $\mathcal{T}[\rho]$ . When control leaves the enclosed block, the pool is released. In its translation we use the auxiliary function **RUNPOOL**. It binds the current continuation  $k$  and creates a fresh pool  $h$ . We run the given computation  $m$  with  $h$  and a continuation where we push a frame that releases the pool onto the current continuation  $k$ . This ensures that we releases the pool when we return normally from the enclosed block.

**Extended Translation Rules:**

$$\begin{aligned}
\mathcal{T}[\text{Catch } \rho] &= \text{CPS } \mathcal{T}[\rho] \text{ Void} \\
\mathcal{S}[\text{try } \{ [r](x, l) \Rightarrow s_0 \} \text{ catch } \{ s \}]_\rho &= \text{RUNCPS}((\Lambda r. \lambda x. \lambda l. \mathcal{S}[s_0]_r) \text{ (CPS } \mathcal{T}[\rho] \mathcal{T}[\tau] \text{)}) (\lambda k. \mathcal{S}[s]_\rho) \text{ (LIFTCPS)} \\
\mathcal{S}[\text{throw}(e, i)]_\rho &= \mathcal{E}[i] \text{ Void } \mathcal{E}[e]
\end{aligned}$$

**Auxiliary Definitions:**

$$\begin{aligned}
\text{RUNCPS} &: \text{CPS} (\text{CPS } R A) A \rightarrow \text{CPS } R A \\
\text{RUNCPS} &= \lambda m. m (\lambda x. \lambda k. k x) \\
\text{LIFTCPS} &: \forall a. \text{CPS } R a \rightarrow \text{CPS} (\text{CPS } R R') a \\
\text{LIFTCPS} &= \Lambda a. \lambda m. \lambda k. \lambda j. m (\lambda x. k x j)
\end{aligned}$$

**Fig. 12.** Translation of  $\Lambda_\rho$  with exceptions.

Evidence terms are functions  $\text{LIFTPOOL } h$  that release the pool  $h$ . Our types make sure that we evaluate the evidence if-and-only-if we non-locally leave the body of the pool. In Section 3.4, evidence was a list of pools. Here, evidence still contains a list of pools, but this list is hidden in the closure environment of the evidence. Evidence composition conceptually concatenates these lists.

The **open** statement opens a file and registers it in the pool. The **readln** statement uses a primitive to read from a file. We require evidence that the pool is live, *i.e.* on the runtime stack, but do not have to actually use it. As we have seen in Section 3.7 its existence is enough to assert that accessing the file is safe.

*Example 4.* Let us consider a simplified version of the motivating example (Section 2.1). The example on the left translates to the term in **System F** on the right. It has type  $\text{CPS Void Int}$ .

|                                                                                                        |                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> pool {   [r1](p1: Pool r1, l1: r1 ⊆ T) ⇒     val f = open(p1, "input", 0);     return 0 } </pre> | <pre> λk.   let h = createPool ();   (Λr1. λp1. λl1. λk1.     let f = openFile p1 "input";     k1 0) Void h   (Λa. λm. λk. releasePool h; m k)   (λx. releasePool h; k x) </pre> |
|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

This term can be normalized to the following:

```
λk. let h = createPool (); let f = openFile h "input"; releasePool h; k 0
```

**4.3 Exceptions**

In this subsection, we present the translation of exceptions. Whereas in the operational semantics (Section 3.5) we have divided the stack into regions with markers, we now have multiple stacks, *i.e.* continuations. We have seen that

evidence terms contained exactly the list of markers we have to unwind when we throw to a handler. Now we take advantage of this fact and let the evidence be the unwinding action itself. Figure 12 presents the translation of exceptions. It is different from the translation to double-barrelled CPS [17, 29], where functions only ever get exactly two continuations. Under our translation to iterated CPS functions can receive any number of continuations.

To support aborting the computation, we instantiate the answer type  $r$  of the translated body  $s_0$  to be the type  $\text{Cps } \mathcal{T}[\rho] \mathcal{T}[\tau]$ . This adds another layer of CPS and one additional (curried) continuation argument. In the translation of **try** ... **catch** ... statements, we use `RUNCPS`. It runs the given computation  $m$  with an additional continuation which is initially empty. The evidence  $l$  lifts the given computation from the inner region to the outer region. It will be bound to `LIFTCPs` which pushes the current continuation onto the next one.

A `Catch`  $\rho$  is a CPS expression that aborts the computation. That is, the handler  $(\lambda k. \mathcal{S}[s]_\rho)$  discards the current continuation  $k$ . In the translation of statement **throw**( $e, i$ ), we call the provided evidence  $i$  and then the handler  $e$ . Running the evidence lifts the handler into the correct region, making it compatible with the current answer type. It is safe for the handler to discard the continuation  $k$ , since all cleanup actions contained in  $k$  are run by the evidence.

*Example 5.* Let us consider the example from Section 2.2. The example on the left translates to the resulting term of type `Cps Void Int` on the right.

|                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> try { [r1](e1 : Catch r1, l1 : r1 <math>\sqsubseteq</math> T) <math>\Rightarrow</math>   safeDiv[r1](5, 0, e1) } catch {   return 0 } </pre> | <pre> (<math>\Lambda r_1. \lambda e_1. \lambda l_1.</math>   safeDiv <math>r_1</math> 5 0 <math>e_1</math> ) (Cps Void Int)   (<math>\lambda k_1. \lambda k_2. k_2</math> 0)   (<math>\Lambda a. \lambda m. \lambda k. \lambda j. m (\lambda x. k x j)</math>)   (<math>\lambda x. \lambda k. k x</math>) </pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

The resulting System F term can be beta reduced and eta expanded to:

$$\lambda k_2. \text{safeDiv} (\text{Cps Void Int}) \ 5 \ 0 \ (\lambda k_1. \lambda k_2. k_2 \ 0) \ (\lambda x. \lambda k. k \ x) \ k_2$$

We instantiate the answer type  $r$  of `safeDiv` with  $r_1$ , which itself is instantiated with `Cps Void Int`. The return type is `Cps (Cps Void Int) Int` and our program now receives two continuations. To abort, the exception handler discards the first one (*i.e.*,  $k_1$ ) and returns 0 to the second one (*i.e.*,  $k_2$ ).

#### 4.4 Combining Resource Pools and Exceptions

Well-typed programs in  $\Lambda_\rho$  translate to well-typed programs in System F.

##### Theorem 3 (Well-typedness of Translated Terms).

*If  $\Gamma \vdash \rho \vdash s : \tau$ , then  $\mathcal{T}[\Gamma] \vdash \mathcal{S}[s]_\rho : (\mathcal{T}[\tau] \rightarrow \mathcal{T}[\rho]) \rightarrow \mathcal{T}[\rho]$*

*Proof (Proof).*

Straightforward induction over the typing derivation.

The translation of exception handlers in Section 4.3 automatically interacts correctly with the evidence terms we have defined for resource pools in Section 4.2: We clear a pool exactly when an exception is thrown across it. This is because we have chosen the translation of evidence to be a concrete computation that moves from one region to another one.

*Example 6.* The following is an extended example where we combine resource pools and exceptions in a more complicated way. The program splits a large input file into smaller files of 100 lines each.

```
try { [r1](stop : Catch r1, l1 : r1  $\sqsubseteq$  Top)  $\Rightarrow$ 
  withFile[r1]("input", { [r2](in: File r2, l2 : r2  $\sqsubseteq$  r1)  $\Rightarrow$ 
    def copyFile(target : String) at r2 {
      withFile[r2](target, { [r3](out: File r3, l3 : r3  $\sqsubseteq$  r2)  $\Rightarrow$ 
        def copyLine() at r3 {
          if (isEOF(in, l3)) { throw(stop, l3  $\oplus$  l2) }
          else { writeln(out, readln(in, l3), 0) }
        };
        def innerLoop(toCopy : Int) at r3 {
          if (toCopy > 0) { copyLine(); innerLoop(toCopy - 1) }
        };
        innerLoop(100)
      })
    };
    def loop(n : Int) at r2 { copyFile("output" ++ n); loop(n + 1) };
    loop(0)
  })
} catch { return () }
```

When we encounter the end of the input file, we simply throw an exception to terminate the program. We can be confident that all resources will be properly cleaned up and so fearlessly use exceptions to structure control flow. The outer loop, for example never returns. It is terminated by throwing an exception. This program, after CPS translation, manually applying contification [17], and beta reduction, reduces to the code in Figure 13.

Our CPS translation of both regions and control enables aggressive optimization. For example, at the end of the input file, we immediately release both pools and return. Since we only apply well-known optimizations on functional programs, we can be certain of their correctness without having to reason explicitly about resources nor control effects nor their combination. The overall correctness of the optimized result rests on the correctness of our CPS translation.

## 4.5 Simulation of the Machine Semantics by the CPS translation

In Section 3, we defined an operational semantics for  $\Lambda_\rho$ . In this section we defined a CPS translation for  $\Lambda_\rho$ . We now show that the two behave the same. This entails that the operational properties from Section 3 carry over to the

```

 $\lambda k.$ 
  let  $p_2 = \text{createPool}()$ ;
  let  $in = \text{openFile } p_2 \text{ "input"};$ 
  let rec loop  $n = (\lambda k_1.$ 
    let  $p_3 = \text{createPool}()$ ;
    let  $out = \text{openFile } p_3 \text{ ("output" ++ } n);$ 
    let rec innerLoop toCopy = ( $\lambda k_2.$ 
      if ( $toCopy > 0$ )
      then if isEOF( $in$ )
        then releasePool  $p_3$ ; releasePool  $p_2$ ; ( $\lambda k_4. k_4 0$ )
        else let  $line = \text{readLine } in$ ; writeLine  $out \text{ } line$ ; innerLoop ( $toCopy - 1$ )
      else releasePool  $p_3$ ; loop ( $n + 1$ )
    );
    innerLoop 100
  );
  loop 0  $k$ 

```

**Fig. 13.** Result of translating Example 6 to CPS.

CPS translation and that optimization via beta reduction is sound. To show preservation of semantics, we extend our translation to machine states [4, 15]. We translate statements to terms and stacks to evaluation contexts in **System F**. We define the translation  $\mathcal{M}[\![ \cdot ]\!]$  of machine states as the plugging of the translation of the statement into the translation of the stack. The full translation is available in a separate technical report [27].

We show that for each step the machine takes, there is a corresponding (possibly empty) sequence of steps between the translated terms.

**Theorem 4 (Simulation).**

If  $M \rightarrow M'$ , then  $\mathcal{M}[\![ M ]\!] \rightarrow^* \mathcal{M}[\![ M' ]\!]$ .

*Proof (Proof).*

By considering each case of the stepping relation. The *(throw)* step needs its own lemma, which we show by induction on possible evidence expressions.

Since for simulation we are only interested in operational behavior, we target the untyped lambda calculus (with primitives for file management) instead of **System F**. The translation of statements is the same as  $\mathcal{S}[\![ s ]\!]_\rho$  in Figures 10, 11, and 12, but we erase all type annotations, type abstractions, and type applications. There is no harm in doing so, since our target is in CPS where the evaluation order is explicit.

While the operational semantics given in Section 3 discards frames during unwinding, for our proof of simulation we have to retain them. We do so in a third component of the machine state  $(\text{throw}(h, w) \parallel K \parallel H)$ : the stack trace  $H$ . This is necessary because the CPS translation discards the whole continuation in one step, while the operational semantics unwinds the stack frame-by-frame.

We translate the empty stack to a special primitive function **done**, which will return the overall result of the program. It is called exactly once, when the machine is in its final state and we return to the empty stack.

*Example 7.* Pools are created and released exactly when they would be in the operational semantics. As an illustration, consider the following sequence of machine steps where we unwind a pool frame:

```

< throw(h1, (po h2 :: •)) || #poolh2 {□} :: #catchh1 {□} {return 1} :: • > →
< throw(h1, (po h2 :: •)) || #poolh2 {□} :: #catchh1 {□} {return 1} :: • || • > →
< throw(h1, •) || #catchh1 {□} {return 1} :: • || #poolh2 {□} :: • > →
< return 1 || • >

```

The first step (*throw*) goes from normal execution to the unwinding state which accumulates frames in its third component. The next two steps are (*free*) and (*catch*). In CPS, we can observe the same program trace:

```

((LIFTPOOL h2) (λk1. λk2. k2 1)) (λx. releasePool h2; (λx. λk. k x) x) done →
(λk. releasePool h2; (λk1. λk2. k2 1) k) (λx. releasePool h2; (λx. λk. k x) x) done →
(λk1. λk2. k2 1) (λx. releasePool h2; (λx. λk. k x) x) done →
(λk2. k2 1) done

```

*Example 8.* Although we do not have any markers generated at runtime, the CPS translation exactly mimics the behavior of the operational semantics, which does have them. Consider another example, where we throw an exception to an outer handler. The steps are (*throw*), (*forward*), and (*catch*).

```

< throw(h1, (h2 :: •)) || #catchh2 {□} {return 2} :: #catchh1 {□} {return 1} :: • > →
< throw(h1, (h2 :: •)) || #catchh2 {□} {return 2} :: #catchh1 {□} {return 1} :: • || • > →
< throw(h1, •) || #catchh1 {□} {return 1} :: • || #catchh2 {□} {return 2} :: • > →
< return 1 || • >

```

In CPS, we start out with three continuations, then we push the first one onto the second one, then the exception handler discards both in one step:

```

(LIFTCP (λk1. λk2. k2 1)) (λx. λk. k x) (λx. λk. k x) done →
(λk. λj. (λk1. λk2. k2 1) (λy. k y j)) (λx. λk. k x) (λx. λk. k x) done →
(λk1. λk2. k2 1) (λy. (λx. λk. k x) y (λx. λk. k x)) done →
(λk2. k2 1) done

```

The CPS translation exhibits the same behavior as the operational semantics. It simulates the generative semantics of exceptions. Remarkably, it does not need any runtime support for markers on the stack to do so. Indeed, in CPS there is no stack!

## 5 Related Work

Out of the large body of work on regions, the one most closely related, and indeed which has been the basis of our work, is the one by Kiselyov and Shan [19], which in turn is based on work of Fluet and Morrisett [12]. Kiselyov and Shan provide a library for region-based resource management in Haskell. They demonstrate how types, regions, and subregioning evidence are inferred, which we do not discuss. They deal with builtin Haskell exceptions, but leave a formal proof to future work. We go further, and add exceptions as a language feature, and prove region- and exception safety. Moreover, we present a CPS translation of these features.

Crary et al. [9] present a language with *dynamic* regions, where regions do not have to be nested, resource access is safe, but resource cleanup is not automatic but explicit. Their language is presented in CPS. Indeed, to quote Fluet et al. [13]: “Dynamic regions are not restricted to LIFO lifetimes and can be treated as first-class objects. They are particularly well suited for iterative computations, CPS-based computations, and event-based servers where lexical regions do not suffice.” We present a CPS translation of *lexical* regions where resources are automatically released, even when an exception is thrown.

Clearly also related is the line of work on monadic encapsulation of state [20, 23, 28]. The most recent work in this line [31] presents a mechanized proof of a number of equivalences in the presence of encapsulated mutable state. We merely prove that references are not used outside of their region, but do so in the presence of exceptions.

Kiselyov and Ishii [18] present a Haskell library for effect handlers based on a variant of the free monad in Haskell. Their library supports user-defined effects and handlers and they provide a range of pre-defined effects like exceptions, non-determinism, and state. They also discuss a region effect for safe and automatic allocation and release of resources, which correctly works in the presence of the exception effect. Other effects, like non-determinism, are explicitly ruled out by the type system when they would be used across a resource delimiter. They reify the structure of the program as a free monad and then write interpreters over this structure, whereas we translate programs to CPS. Moreover we provide a proof of region- and exception safety, which is out of scope of their work.

Leijen [21] reports on an extension of the programming language Koka with support for resources and finalization. They support general effect handlers, while we merely discuss the special case of exceptions. Their approach requires sophisticated modification of the language runtime, whereas our approach can be explained as a translation to pure System F. They allow for more complex finalization patterns, where users explicitly run the finalizers of a resumption. This is to avoid running finalizers on linearly used resumptions, a problem that we completely side-step by only discussing exceptions.

Ahman and Bauer [1] present an approach to resource management: Runners. They guarantee that cleanup actions are run exactly once. We offer the same guarantee. We present an operational semantics that relates resource man-



agement to the stack and a translation of programs to CPS. Their denotational semantics translates programs to essentially a free monad.

Thielecke [29] compares different control constructs by their translation to double-barrelled CPS where functions receive exactly two continuations. In contrast, under our iterated CPS translation functions can receive any number of continuations. Moreover, even in the case where we pass two continuations, there is a difference. Whereas in double-barrelled CPS translated terms have type:

$$(\llbracket A \rrbracket \rightarrow \text{Ans}) \rightarrow (\llbracket B \rrbracket \rightarrow \text{Ans}) \rightarrow \text{Ans}$$

Under our iterated CPS translation such terms would have type:

$$(\llbracket A \rrbracket \rightarrow (\llbracket B \rrbracket \rightarrow \text{Ans}) \rightarrow \text{Ans}) \rightarrow (\llbracket B \rrbracket \rightarrow \text{Ans}) \rightarrow \text{Ans}$$

Their work is neither concerned with resources nor multiple different exception handlers.

Thielecke [30] studies the connection between control effects and continuation passing. His work introduces some of the ideas presented in this paper: regions are answer types, region polymorphism is answer-type polymorphism, and effect masking introduces a fresh region to delimit the extent of control effects. We expand upon his work in several ways: Instead of a single control operator `call/cc`, we consider a language with multiple layers of exceptions and resources. Therefore, on the type level, we have subregioning evidence between nested regions, and on the term level, we translate to iterated CPS.

Our iterated CPS translation of exceptions is closely related to the one presented by Schuster et al. [26]. However, they do not support effect-polymorphic functions. Our translation to System F is similar to the one for effect handlers sketched in Appendix B of Hillerström et al. [15].

## 6 Conclusion

We presented  $\Lambda_\rho$ , a language with first-class functions, regions, resources, and exceptions. Its type system guarantees safe access to resources and safe use of exceptions. We then presented a CPS translation that preserves these guarantees.

We view regions as describing runtime stacks. This view is very much in line with recent work on effect handlers. One does wonder if our approach scales to more general control effects, which do not discard the current continuation, and perhaps even use it multiple times. This is the subject of ongoing investigation.

## Acknowledgments

The work on this project was supported by the Deutsche Forschungsgemeinschaft (DFG – German Research Foundation) – project number DFG-448316946.

## References

- [1] Ahman, D., Bauer, A.: Runners in action. In: Müller, P. (ed.) *Programming Languages and Systems*, pp. 29–55, Springer International Publishing, Cham (2020)
- [2] Appel, A.W.: *Compiling with Continuations*. Cambridge University Press, New York, NY, USA (1992), ISBN 0-521-41695-7
- [3] Bertot, Y., Castéran, P.: *Interactive Theorem Proving and Program Development, Coq’Art: The Calculus of Inductive Constructions*. Springer-Verlag (2004)
- [4] Biernacki, D., Piróg, M., Polesiuk, P., Sieczkowski, F.: Abstracting algebraic effects. *Proc. ACM Program. Lang.* **3**(POPL), 6:1–6:28 (Jan 2019)
- [5] Biernacki, D., Piróg, M., Polesiuk, P., Sieczkowski, F.: Binders by day, labels by night: Effect instances via lexically scoped handlers. *Proc. ACM Program. Lang.* **4**(POPL) (Dec 2019), <https://doi.org/10.1145/3371116>
- [6] Brachthäuser, J.I., Schuster, P., Ostermann, K.: Effects as capabilities: Effect handlers and lightweight effect polymorphism. *Proc. ACM Program. Lang.* **4**(OOPSLA) (Nov 2020), <https://doi.org/10.1145/3428194>
- [7] Brady, E.: *Idris 2: Quantitative type theory in action*. Tech. rep., University of St Andrews, Scotland, UK (2020), URL <https://www.type-driven.org.uk/edwinb/papers/idris2.pdf>
- [8] Cong, Y., Osvald, L., Essertel, G.M., Rompf, T.: Compiling with continuations, or without? whatever. *Proc. ACM Program. Lang.* **3**(ICFP), 79:1–79:28 (Jul 2019), <https://doi.org/10.1145/3341643>
- [9] Crary, K., Walker, D., Morrisett, G.: Typed memory management in a calculus of capabilities. In: *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, p. 262–275, POPL ’99, Association for Computing Machinery, New York, NY, USA (1999), <https://doi.org/10.1145/292540.292564>
- [10] Danvy, O.: *On evaluation contexts, continuations, and the rest of computation* (02 2004)
- [11] Danvy, O., Filinski, A.: Abstracting control. In: *Proceedings of the Conference on LISP and Functional Programming*, pp. 151–160, ACM, New York, NY, USA (1990)
- [12] Fluet, M., Morrisett, G.: Monadic regions. In: *Proceedings of the Ninth ACM SIGPLAN International Conference on Functional Programming*, p. 103–114, ICFP ’04, Association for Computing Machinery, New York, NY, USA (2004), <https://doi.org/10.1145/1016850.1016867>
- [13] Fluet, M., Morrisett, G., Ahmed, A.: Linear regions are all you need. In: Sestoft, P. (ed.) *Programming Languages and Systems*, pp. 7–21, Springer Berlin Heidelberg, Berlin, Heidelberg (2006)
- [14] Grossman, D., Morrisett, G., Jim, T., Hicks, M., Wang, Y., Cheney, J.: Region-based memory management in cyclone. In: *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation*, p. 282–293, PLDI ’02, Association for Computing Machinery, New York, NY, USA (2002), <https://doi.org/10.1145/512529.512563>

- [15] Hillerström, D., Lindley, S., Atkey, B., Sivaramakrishnan, K.: Continuation passing style for effect handlers. In: *Formal Structures for Computation and Deduction, LIPIcs*, vol. 84, Schloss Dagstuhl–Leibniz-Zentrum für Informatik (2017)
- [16] Hillerström, D., Lindley, S., Atkey, R.: Effect handlers via generalised continuations. *Journal of Functional Programming* **30**, e5 (2020), <https://doi.org/10.1017/S0956796820000040>
- [17] Kennedy, A.: Compiling with continuations, continued. In: *Proceedings of the International Conference on Functional Programming*, pp. 177–190, ACM, New York, NY, USA (2007)
- [18] Kiselyov, O., Ishii, H.: Freer monads, more extensible effects. In: *Proceedings of the Haskell Symposium*, pp. 94–105, ACM, New York, NY, USA (2015)
- [19] Kiselyov, O., Shan, C.c.: Lightweight monadic regions. In: *Proceedings of the Haskell Symposium, Haskell '08*, ACM, New York, NY, USA (2008)
- [20] Launchbury, J., Peyton Jones, S.L.: Lazy functional state threads. In: *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation*, p. 24–35, PLDI '94, Association for Computing Machinery, New York, NY, USA (1994), <https://doi.org/10.1145/178243.178246>
- [21] Leijen, D.: Algebraic effect handlers with resources and deep finalization. Tech. Rep. MSR-TR-2018-10, Microsoft Research (April 2018)
- [22] Levy, P.B., Power, J., Thielecke, H.: Modelling environments in call-by-value programming languages. *Information and Computation* **185**(2), 182–210 (2003)
- [23] Moggi, E., Sabry, A.: Monadic encapsulation of effects: a revised approach (extended version). *Journal of Functional Programming* **11**(6), 591–627 (Nov 2001)
- [24] Reynolds, J.C.: Definitional interpreters for higher-order programming languages. In: *Proceedings of the ACM annual conference*, pp. 717–740, ACM, New York, NY, USA (1972)
- [25] Schuster, P., Brachthäuser, J.I.: Typing, representing, and abstracting control. In: *Proceedings of the Workshop on Type-Driven Development*, pp. 14–24, ACM, New York, NY, USA (2018), <https://doi.org/10.1145/3240719.3241788>
- [26] Schuster, P., Brachthäuser, J.I., Ostermann, K.: Compiling effect handlers in capability-passing style. *Proc. ACM Program. Lang.* **4**(ICFP) (Aug 2020), <https://doi.org/10.1145/3408975>
- [27] Schuster, P., Brachthäuser, J.I., Ostermann, K.: Region-based resource management and lexical exception handlers in continuation-passing style (technical report). Tech. rep., University of Tübingen, Germany (01 2022), <https://se.informatik.uni-tuebingen.de/publications/schuster22region/>
- [28] Semmelroth, M., Sabry, A.: Monadic encapsulation in ml. In: *Proceedings of the Fourth ACM SIGPLAN International Conference on Functional Programming*, p. 8–17, ICFP '99, Association for Computing Machinery, New York, NY, USA (1999), <https://doi.org/10.1145/317636.317777>

- [29] Thielecke, H.: Comparing control constructs by double-barrelled cps. *Higher Order Symbol. Comput.* **15**(2-3), 141–160 (sep 2002), <https://doi.org/10.1023/A:1020887011500>
- [30] Thielecke, H.: From control effects to typed continuation passing. In: *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, p. 139–149, POPL '03, Association for Computing Machinery, New York, NY, USA (2003), <https://doi.org/10.1145/604131.604144>
- [31] Timany, A., Stefanescu, L., Krogh-Jespersen, M., Birkedal, L.: A logical relation for monadic encapsulation of state: Proving contextual equivalences in the presence of runst. *Proc. ACM Program. Lang.* **2**(POPL) (Dec 2017), <https://doi.org/10.1145/3158152>
- [32] Tofte, M., Birkedal, L., Elsmann, M., Hallenberg, N., Sestoft, P.: Programming with regions in the ml kit (for version 4) (10 2001)
- [33] Tofte, M., Talpin, J.P.: Region-based memory management. *Inf. Comput.* **132**(2), 109–176 (Feb 1997), <https://doi.org/10.1006/inco.1996.2613>
- [34] Xie, N., Brachthäuser, J.I., Hillerström, D., Schuster, P., Leijen, D.: Effect handlers, evidently. *Proc. ACM Program. Lang.* **4**(ICFP) (Aug 2020), <https://doi.org/10.1145/3408981>
- [35] Zhang, Y., Myers, A.C.: Abstraction-safe effect handlers via tunneling. *Proc. ACM Program. Lang.* **3**(POPL), 5:1–5:29 (Jan 2019)
- [36] Zhang, Y., Salvaneschi, G., Beightol, Q., Liskov, B., Myers, A.C.: Accepting blame for safe tunneled exceptions. In: *Proceedings of the Conference on Programming Language Design and Implementation*, pp. 281–295, ACM, New York, NY, USA (2016)

**Open Access** This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

